

COMPUTER VISION 5주차 · Local Feature Detection

수작업 필터와 학습 필터를 결합한 특징점 검출: Key.Net

미분 필터를 CNN 첫 층에 고정해 넣은 특징점 검출기

Barroso-Laguna, Riba, Ponsa, Mikolajczyk. Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters. ICCV 2019.

김아영 | 데이터사이언스융합학과 | 2025713982

01 연구 배경: 학습 기반 검출기의 한계

딥러닝 검출기는 "같은 지점을 다시 찾는 능력(반복성)"에서 기존 수작업 검출기 대비 뚜렷한 우위 없음

구분	대표 방법	작동 방식
수작업 검출기	Harris(1988), Hessian(1978), SIFT(2004), KAZE(2012)	이미지를 미분한 값을 조합해 코너-블록을 찾음
학습 검출기	FAST(2006), TILDE(2015), LIFT(2016), SuperPoint, LF-Net(2018)	데이터로 학습해 특징점을 찾음

논문이 지적한 문제

- 특징 설명(기술자): CNN 도입 후 성능 크게 향상
- 특징점 탐색(검출기): 반복성 개선 폭 제한적
- 특히 사진 크기 변화에 취약

논문의 아이디어

- 기존 검출기(DoG, Harris-Laplace 등)는 이미지 미분을 조합해 계산
- 이는 CNN의 합성곱 연산과 구조적으로 유사
- 미분 필터를 CNN에 직접 투입 → 적은 층으로도 고성능 검출기 구현 가능

Key.Net 전체 구조

고정된 미분 필터 10개 → 학습 필터 → 3가지 크기 결과 결합 순으로 픽셀별 "특징점 점수" 산출

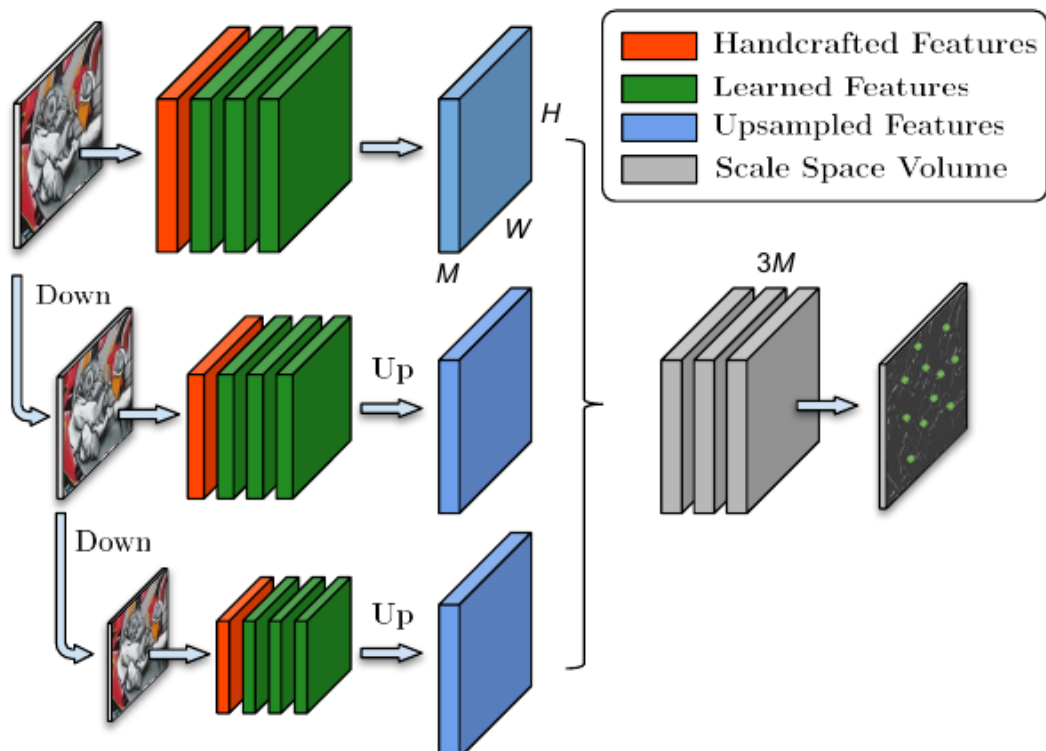


Figure 1

(1) 입력 피라미드 (Down)

입력 사진을 흐리게 한 뒤 1.2배씩 줄여 3가지 크기 생성

(2) 수작업 필터 (빨강)

미분 필터 10개. 값은 고정 (학습 대상 아님)

(3) 학습 필터 (초록)

5×5 필터 8개 + 배치 정규화 + ReLU를 한 묶음으로 3회 적층
3가지 크기가 동일 필터 공유함

(4) 합치기 (파랑·회색)

결과를 원래 크기로 키워 이어 붙이고, 마지막 필터 하나로 점수 지도 생성

03 수작업 필터 ① 1차 미분 5개

1차 미분 필터 5개(I_x , I_y , I_x^2 , I_y^2 , $I_x \cdot I_y$) = Harris 코너 검출기 행렬의 구성 요소

필터	의미
I_x , I_y	가로·세로 방향으로 밝기가 얼마나 바뀌는지
I_x^2 , I_y^2 , $I_x \cdot I_y$	Harris 행렬 $\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix}$ 의 성분

논문: "Harris 검출기의 두 번째 모멘트 행렬처럼 $I_x \cdot I_y$, I_x^2 , I_y^2 를 계산한다."

미분은 가우시안을 미분한 필터로 계산하며, 잡음에 약한 3차 이상은 쓰지 않는다.

Harris 검출 원리

- 창 안의 I_x^2 , I_y^2 , $I_x \cdot I_y$ 를 합산하여 2x2 행렬 구성
- 두 고유값이 모두 작으면 **평탄**, 하나만 크면 **엣지**, 둘 다 크면 **코너**
- 점수 $R = \det - \alpha \cdot \text{trace}^2$. $\rightarrow R$ 이 크면 코너

계산 예시: Sobel 연산자로 코너 판별

오른쪽 아래만 밝은 코너 이미지 (0=어둠, 1=밝음)

0	0	0	0	0
0	0	0	0	0
0	0	1	1	1
0	0	1	1	1
0	0	1	1	1

- Sobel로 I_x , I_y 를 구하고
- 가운데 3x3 창에서 더하면
- $\sum I_x^2 = 52$, $\sum I_y^2 = 52$
- $\sum I_x \cdot I_y = 16$

창	행렬	고유값	판정
평탄	$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$	0, 0	평탄
세로 엣지	$\begin{bmatrix} 96 & 0 \\ 0 & 0 \end{bmatrix}$	96, 0	엣지
코너	$\begin{bmatrix} 52 & 16 \\ 16 & 52 \end{bmatrix}$	68, 36	코너

04 수작업 필터 ② 2차 미분 5개

2차 미분 필터 5개(l_{xx} , l_{yy} , l_{xy} , $l_{xx} \cdot l_{yy}$, l_{xy}^2) = Hessian 행렬의 구성요소로, 점(블롭) 탐지에 활용

필터	의미
l_{xx} , l_{yy} , l_{xy}	2차 미분 = 밝기 변화가 얼마나 휘는지. Hessian 행렬 [l_{xx} l_{xy} ; l_{xy} l_{yy}]의 성분
$l_{xx} \cdot l_{yy}$, l_{xy}^2	Hessian 행렬식 $\det = l_{xx} \cdot l_{yy} - l_{xy}^2$ 의 두 항

논문: "Hessian 검출기는 Hessian 행렬의 행렬식을 사용하므로 $l_{xx} \cdot l_{yy}$ 와 l_{xy}^2 를 추가한다."

논문 $\Rightarrow$ 강의 내용과의 관계

블롭 검출에 쓴 라플라시안 $\nabla^2 g = \partial^2 g / \partial x^2 + \partial^2 g / \partial y^2$

- = Hessian 행렬의 대각선 두 값(l_{xx} , l_{yy})의 합
- Hessian은 여기에 대각선 방향 변화(l_{xy})까지 포함한 것

계산 예시: 밝은 점 vs 세로 엣지

0	0	0	0	0
0	1	2	1	0
0	2	4	2	0
0	1	2	1	0
0	0	0	0	0

밝은 점의 중심

- $l_{xx} = 2 - 2 \times 4 + 2 = -4$
- $l_{yy} = -4$, $l_{xy} = 0$
- $\det = (-4)(-4) - 0 = 16$

0	4	4
0	4	4
0	4	4

세로 엣지

- $l_{xx} = -4$, $l_{yy} = 0$, $l_{xy} = 0$
- $\det = 0$

→ 점 모양은 $\det$ 가 크고, 엣지는 $\det$ 가 0임.

2차 미분 필터가 점 모양 구조를 구별하는 단서로 작용

05 학습 필터와 다중 스케일 피라미드

학습 필터는 3묶음, 입력 크기는 3단계 사용. 3단계 이후에는 성능이 거의 오르지 않아 계산 비용을 고려해 3단계 채택

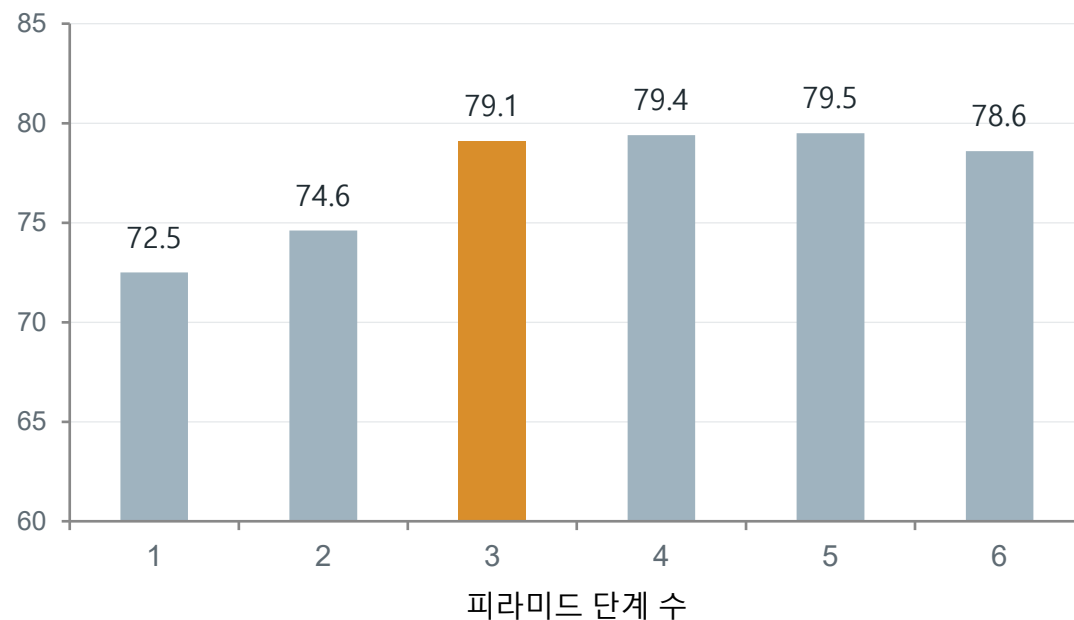
학습 필터 설정

- 한 묶음 = 5×5 합성곱 필터 8개 + 배치 정규화 + ReLU
- 묶음을 3개 쌓는다. 전체 학습 파라미터는 약 5.9천 개이다.
- 미분 필터를 고정해 넣어 학습할 값의 수를 줄이고, 학습을 안정적으로 생성

입력 피라미드 설정

- 입력을 흐리게 한 뒤 1.2배씩 줄여 3단계로 구성
- 목적: 한 번의 계산으로 작은 크기 변화에도 강하게 만들기
- 단계 추가 = 한 번에 보는 영역(수용 영역) 확대 효과

피라미드 단계 수에 따른 반복성 (%)



1단계 72.5% → 3단계 79.1%(+6.6%p) → 5단계 79.5%(+7.0%p). 검증용 합성 데이터 기준.

06 학습 방법: 정답 특징점 없이 학습하기

특징점에는 정답 위치가 없으므로, 두 사진 사이의 변환 관계(호모그래피)를 정답으로 삼아 "같은 지점을 찍도록" 학습

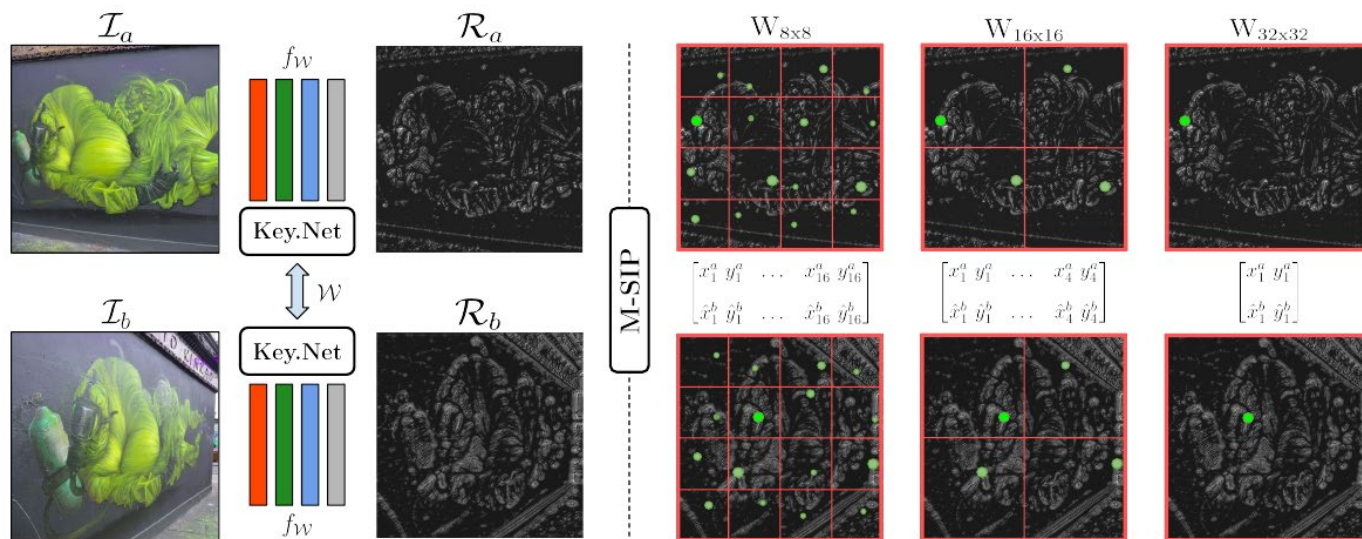


Figure 2. 같은 필터를 공유하는 두 Key.Net이 사진 A, B의 점수 지도를 만든다.

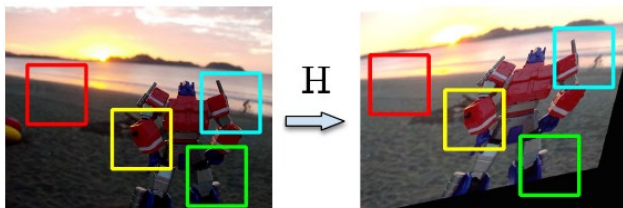


Figure 4. 빨간 영역(무늬 없음)은 제외

학습 원리

- 같은 사진을 변형해 사진 A, B 생성함. 둘 사이의 변환(H)은 기지값
- A에서 찍은 점과, B에서 찍은 점을 H로 옮긴 위치가 일치하도록 학습
- 점수가 높은 점일수록 손실에 크게 반영

학습 데이터	내용
원본	ImageNet 사진에서 192×192 크기 쌍 12,000개 (학습 9,000 / 검증 3,000)
변형	크기 0.5~3.5배, 기울임 ± 0.8 , 회전 $\pm 60^\circ$, 한쪽 사진의 밝기·명암·색조 변경
제외	미분 필터 반응이 낮음(무늬가 없는) 영역은 버림

07 핵심 연산 ① Index Proposal: 최대점을 부드럽게 찾기

"주변보다 큰 점만 남기기(비최대 억제)"는 학습에 쓸 수 없으므로, **창마다 점수 비율로 좌표를 평균 내는 방식(soft-argmax)**으로 대체

(2) 비율 $m = e^{\text{점수}} \div \text{창 안의 } e^{\text{점수}} \text{ 합계}$

(3) 좌표 = 각 칸의 위치 $\times$ 비율 m 의 합

지수(e^x)를 쓰면 가장 큰 점수가 비율을 거의 독차지하므로, 결과 좌표가 최대점 위치에 근접함. 비최대 억제와 유사하나 미분 가능함.

① 창 안의 점수

5	0	0
0	0	0
0	0	4

② $e^{\text{점수}}$

148.4	1	1
1	1	1
1	1	54.6

③ 합계 210.0으로 나눈 비율

0.707	0.005	0.005
0.005	0.005	0.005
0.005	0.005	0.26

④ 좌표 계산

$$x = 1 \times 0.716 + 2 \times 0.014 + 3 \times 0.270$$

$$\approx 1.55$$

→ 최대점(1, 1) 쪽으로 수렴

지수의 밑	1.2	1.4	2.0	e (≈ 2.72)	5.0	7.5
반복성 (%)	77.5	78.4	77.9	79.1	74.6	73.2

- 논문 Table 1(b): 밑이 e일 때 가장 좋았다.
- 밑이 크면 최대점 위치를,
작으면 여러 봉우리의 평균을 반환함

08 핵심 연산 ② 손실 함수와 M-SIP

두 사진에서 찍은 점의 위치 차이를 줄이도록 학습하되, 창 크기를 5가지로 바꿔 모든 크기에서 살아남는 점에 높은 점수를 부여

(4) 손실 = $\sum \alpha \times (A \text{에서 찍은 좌표} - B \text{의 최대점을 H로 옮긴 좌표})^2$ α = 두 점의 점수 합

(5) M-SIP 손실 = $\sum \lambda \times (\text{창 크기별 손실})$ 창 크기 8·16·24·32·40, 가중치 $\lambda = 256 \cdot 64 \cdot 16 \cdot 4 \cdot 1$

식 읽는 법

- A 쪽 좌표는 soft-argmax로, B 쪽은 실제 최대점으로 산출함. 이후 A와 B를 바꿔 재계산 후 합산함
- 큰 창은 좌표 오차가 크므로 λ 를 작게 설정. 논문은 이를 스케일 정규화와 비슷하다고 설명함
- 여러 크기의 창을 함께 쓰면, 네트워크는 모든 크기에서 두드러지는 점에 높은 점수를 주도록 학습함

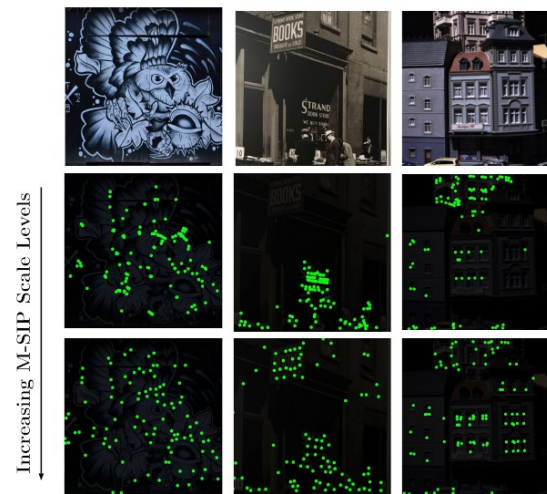


Figure 3. 아래로 갈수록 창이 크다. 옛지·밋밋한 곳의 점은 사라지고 안정적인 점만 남는다.

사용한 창	반복성
8 단독	70.5
8·16	74.6
8·16·24	76.8
8~32	77.6
40 단독	65.7
8~40 전부	79.1

Figure 5 왼쪽 발체 (%)

09 실험 설정과 평가 지표

HPatches 데이터셋에서 13개 기존 검출기와 비교했고, 핵심 지표는 "두 사진에서 같은 점이 다시 검출된 비율(반복성)"

평가 데이터: HPatches

- 116개 사진 묶음 = 시점(촬영 각도) 변화 59개 + 조명 변화 57개
- 각 방법이 두 사진의 겹치는 영역에서 찾은 상위 1,000개 점을 비교
- 비교 대상: SIFT, SURF, Harris-Laplace, KAZE, AKAZE, MSER, SuperPoint, LF-Net 등 13개

학습 환경

- Adam 최적화, 학습률 0.001, 배치 32
- 약 30 epoch, GTX 1080 Ti에서 약 2시간
- 공식 코드 공개 (TensorFlow)

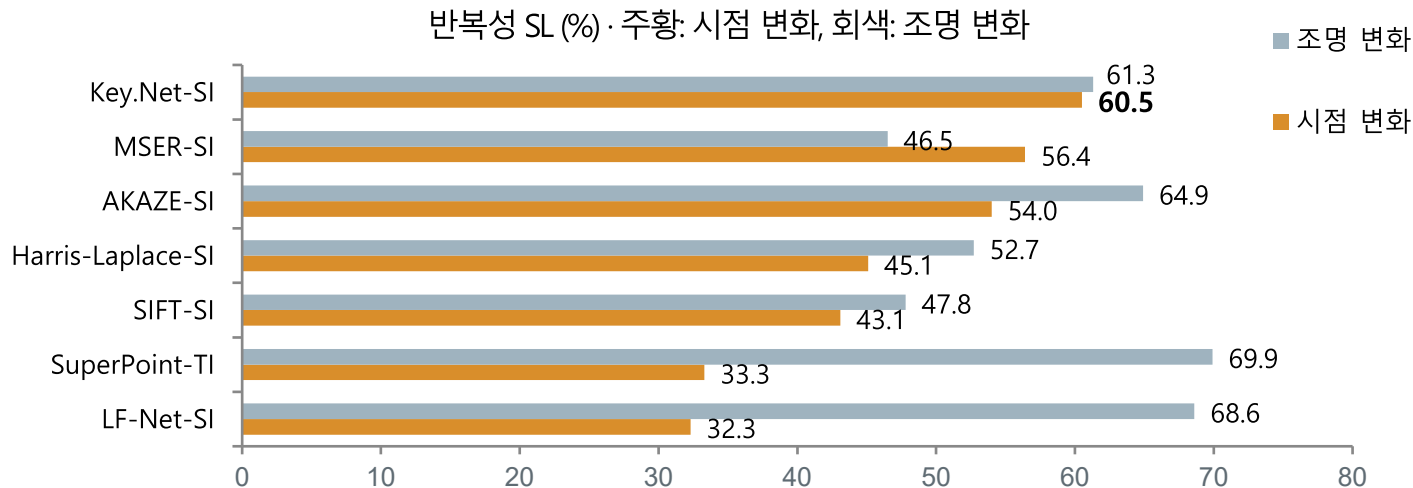
반복성 계산 방법

- **반복성 = 서로 대응된 점의 수 ÷ 두 사진 중 적게 찾은 점의 수**
- 점을 위치와 크기를 가진 원으로 보고, 두 원이 60% 이상 겹치면 대응된 것으로 인정
- 예: 1,000개 중 605개가 대응 → 반복성 60.5%

표기	뜻
-TI	원래 크기에서만 점을 찾음
-SI	여러 크기에서 점을 찾고 크기도 추정함
SL	찾은 위치와 크기를 모두 써서 평가 (실제 성능)
L	크기는 맞았다고 보고 위치만 평가
매칭 점수	같은 기술자(HardNet)를 붙였을 때 올바르게 짝지어진 비율

10 결과 ① 반복성 비교

촬영 각도가 바뀌는 조건에서는 Key.Net이 반복성 1위였지만, 조명만 바뀌는 조건에서는 여러 크기 버전이 타 학습 방법 대비 열위



	Viewpoint					Illumination				
	Repeatability		$\bar{\epsilon}_{IoU}$		S_{range}	Repeatability		$\bar{\epsilon}_{IoU}$		S_{range}
	SL	L	SL	L	SL	SL	L	SL	L	SL
SIFT-SI [5]	43.1	57.6	0.18	0.12	78.6	47.8	60.4	0.18	0.12	84.5
SURF-SI [20]	46.7	60.3	0.18	0.18	24.8	53.0	64.0	0.15	0.11	27.4
FAST-TI [24]	30.4	63.1	0.21	0.10	-	63.6	63.6	0.09	0.09	-
MSER-SI [23]	56.4	62.8	0.12	0.08	503.7	46.5	54.5	0.12	0.10	524.8
Harris-Laplace-SI [34]	45.1	62.0	0.20	0.13	95.9	52.7	62.0	0.17	0.08	90.4
KAZE-SI [21]	53.3	65.7	0.20	0.11	12.5	56.9	65.7	0.12	0.10	12.7
AKAZE-SI [22]	54.0	65.6	0.19	0.10	13.5	64.9	69.1	0.11	0.09	13.6
TILDE-TI [14]	31.0	65.1	0.20	0.15	-	70.4	70.4	0.11	0.11	-
LIFT-SI [7]	43.4	59.4	0.20	0.13	13.3	51.6	65.4	0.18	0.12	13.8
DNet-SI [9]	49.4	62.2	0.21	0.14	11.4	59.1	65.1	0.14	0.13	17.1
TCDET-SI [10]	49.6	61.6	0.23	0.16	6.7	66.9	71.0	0.16	0.15	11.4
SuperPoint-TI [13]	33.3	67.1	0.20	0.17	-	69.9	69.9	0.10	0.10	-
LF-Net-SI [11]	32.3	62.2	0.23	0.12	2.00	68.6	69.1	0.10	0.10	2.0
Tiny-Key.Net-SI	57.8	70.3	0.20	0.12	7.6	56.1	62.8	0.14	0.11	7.6
Key.Net-TI	34.2	71.5	0.20	0.11	-	72.0	72.0	0.10	0.10	-
Key.Net-SI	60.5	73.2	0.19	0.14	7.6	61.3	66.2	0.12	0.10	7.6

M-SIP Region Sizes					Repeatability
$W_{8 \times 8}$	$W_{16 \times 16}$	$W_{24 \times 24}$	$W_{32 \times 32}$	$W_{40 \times 40}$	
✓	-	-	-	-	70.5
✓	✓	-	-	-	74.6
✓	✓	✓	-	-	76.8
✓	✓	✓	✓	-	77.6
-	-	-	-	✓	65.7
-	-	-	✓	✓	71.4
-	-	✓	✓	✓	73.2
-	✓	✓	✓	✓	74.9
✓	✓	✓	✓	✓	79.1

시점 변화: 1위

- Key.Net-SI 60.5%(SL), 73.2%(L)로 가장 높음. (SIFT 대비 17.4%p)
- 필터 1개만 학습하는 Tiny-Key.Net도 상위 3위권

조명 변화: 조건에 따라 다름

- 한 크기만 쓰는 Key.Net-TI는 72.0%로 1위
- 여러 크기 버전(SI)은 61.3%로 낮음
- 저자는 크기 변화가 없는데 크기까지 추정해 생긴 오차로 설명함

위치 정확도

- 점 위치의 평균 오차는 수작업 검출기가 가장 작았음

11 결과 ② 매칭 성능, 효율, 수작업 필터의 역할

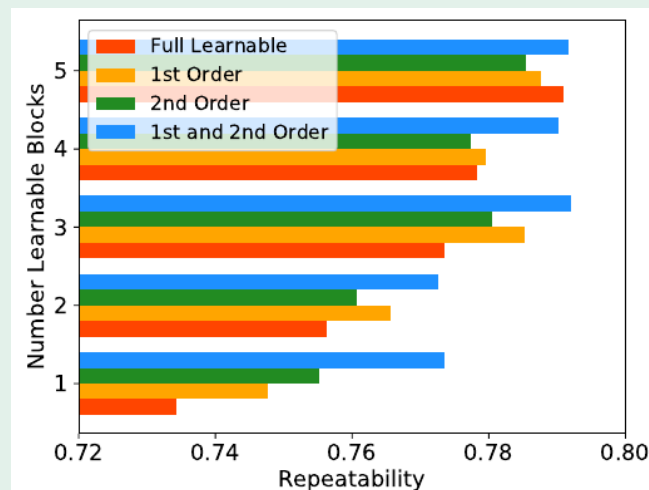
Key.Net은 SuperPoint보다 약 160배 작은 모델로 시점 변화 매칭 점수 1위를 기록. 수작업 필터는 학습 층이 적을 때 효과가 큼

검출기 (+ 같은 기술자)	시점	조명
SIFT	23.2	24.8
AKAZE	36.4	41.4
SuperPoint	37.4	43.0
LF-Net	26.9	43.8
Key.Net	38.4	39.7

매칭 점수(%), Table 3 발췌. 시점 변화 1위, 조명 변화 1위는 LF-Net

모델	학습 파라미터	600×600 처리 시간
SuperPoint	940k	-
LF-Net	39k	-
Key.Net	5.9k	31 ms
Tiny-Key.Net	280	5.7 ms

수작업 필터가 정말 필요한가? (Figure 5 우측)



빨강: 모두 학습
주황: 1차 미분만
초록: 2차 미분만
파랑: 1,2차 모두

세로: 학습 묶음 수
가로: 반복성

- 학습 묶음이 1~2개일 때: 1,2차 미분 필터를 넣은 쪽(파랑)이 뚜렷하게 우세
- 묶음을 늘리면: 모두 학습한 경우(빨강)와 유사한 수준이 됨
- 논문 결론: 미분 필터를 쓰면 더 작은 구조로 같은 성능 달성 가능

논문 결과로 본 강점과 한계

Key.Net은 작고 빠르면서 각도 변화에 강하지만, 조명 변화·위치 정확도에서는 다른 방법보다 약한 부분이 있다.

강점

- 촬영 각도가 바뀌는 조건에서 반복성 1위 (60.5%)
- 학습 파라미터 5.9천 개로 SuperPoint의 약 1/160
- 600×600 사진을 31ms에 처리 (Tiny 버전 5.7ms)
- 논문 결론: 네트워크를 지나치게 복잡하게 만든다고 성능이 좋아지지 않는다.

한계

- 조명만 바뀌는 조건에서 여러 크기 버전의 반복성이 낮음 (61.3%)
- 점 위치의 평균 오차는 수작업 검출기가 더 작음
- 조명 변화 매칭 점수는 LF-Net보다 낮음 (39.7% vs 43.8%)
- 학습 묶음이 많아지면 수작업 필터가 없어도 성능이 비슷해짐

13 최근 동향: RetailGlue (CVPR 2026 Workshop)

최근 소매 선반 연구는 **같은 제품이 반복되는 환경에서 특징점 매칭이 잘 맞지 않는다**고 보고, 제품 단위로 짝을 찾는 방식을 제안함

문제 제기

선반 파노라마 사진은 소매 분석에 필수이지만, 제품 모양이 반복되어 특징점 (local feature) 매칭 기반 이어 붙이기 어려움

방법: 픽셀 단위 대신 제품 단위로 짝 찾기

- ① 제품 검출: YOLOv11l (SKU-110K 데이터로 학습)
- ② 제품 설명: DINOv3 임베딩을 제품의 특징 벡터로 사용
- ③ 제품 짝 찾기: 미세조정된 LightGlue
- ④ 여러 장 이어 붙이기: 그래프 기반, 파노라마 자동 분할

보고된 결과

항목	결과
정확도	F1 88.4% (DINOv3 ViT-B/16 사용 시)
속도 (vs dense 매칭)	RoMa v2보다 20배 빠름
속도 (vs sparse 매칭)	DISK + LightGlue보다 7배 빠름
평가 데이터	RemRetail100_v2 (사진 1,223장, 250개 묶음)

Key.Net과의 비교

- Key.Net: 픽셀 단위 특징점을 "다시 찾는 능력(반복성)"을 높이는 방향
- RetailGlue: 같은 모양이 반복되는 선반에서는 픽셀 단위 대신 제품 단위 대응을 사용

결론

● 미분 필터 + 학습 필터

Harris 행렬과 Hessian 행렬의 재료인 1·2차 미분 10개를 고정 입력으로 넣고, 조합하는 방법만 학습

● 정답 없이 학습하는 방법

최대점 찾기를 soft-argmax 로 대체하고, 두 사진의 변환 관계를 정답으로 여러 크기의 창에서 같은 점을 찍도록 학습

● 결과

각도 변화에서 반복성 1위, 모델 크기 5.9천 개. 조명 변화와 위치 정확도는 향후 과제

Q & A

부록: 논문 Table 2 원문

	Viewpoint					Illumination				
	Repeatability		$\bar{\epsilon}_{IoU}$		S_{range}	Repeatability		$\bar{\epsilon}_{IoU}$		S_{range}
	SL	L	SL	L	SL	SL	L	SL	L	SL
SIFT-SI [5]	43.1	57.6	<u>0.18</u>	0.12	78.6	47.8	60.4	0.18	0.12	84.5
SURF-SI [20]	46.7	60.3	<u>0.18</u>	0.18	24.8	53.0	64.0	0.15	0.11	27.4
FAST-TI [24]	30.4	63.1	0.21	<u>0.10</u>	-	63.6	63.6	0.09	<u>0.09</u>	-
MSER-SI [23]	56.4	62.8	0.12	0.08	503.7	46.5	54.5	0.12	0.10	524.8
Harris-Laplace-SI [34]	45.1	62.0	0.20	0.13	<u>95.9</u>	52.7	62.0	0.17	0.08	<u>90.4</u>
KAZE-SI [21]	53.3	65.7	0.20	0.11	12.5	56.9	65.7	0.12	0.10	12.7
AKAZE-SI [22]	54.0	65.6	0.19	<u>0.10</u>	13.5	64.9	69.1	0.11	<u>0.09</u>	13.6
TILDE-TI [14]	31.0	65.1	0.20	0.15	-	<u>70.4</u>	70.4	0.11	0.11	-
LIFT-SI [7]	43.4	59.4	0.20	0.13	13.3	51.6	65.4	0.18	0.12	13.8
DNet-SI [9]	49.4	62.2	0.21	0.14	11.4	59.1	65.1	0.14	0.13	17.1
TCDET-SI [10]	49.6	61.6	0.23	0.16	6.7	66.9	<u>71.0</u>	0.16	0.15	11.4
SuperPoint-TI [13]	33.3	67.1	0.20	0.17	-	69.9	69.9	<u>0.10</u>	0.10	-
LF-Net-SI [11]	32.3	62.2	0.23	0.12	2.00	68.6	69.1	<u>0.10</u>	0.10	2.0
Tiny-Key.Net-SI	<u>57.8</u>	70.3	0.20	0.12	7.6	56.1	62.8	0.14	0.11	7.6
Key.Net-TI	34.2	<u>71.5</u>	0.20	0.11	-	72.0	72.0	<u>0.10</u>	0.10	-
Key.Net-SI	60.5	73.2	0.19	0.14	7.6	61.3	66.2	0.12	0.10	7.6

References

주 논문

Key.Net: Barroso-Laguna, A., Riba, E., Ponsa, D., Mikolajczyk, K., "Key.Net: Keypoint Detection by Handcrafted and Learned CNN Filters", ICCV 2019.

최근 동향

RetailGlue: Öztüner, A., Çelik, A., Yalçiner, İ. Ş., Calap, S., "RetailGlue: Semantic Product-Level Image Stitching for Retail Shelf Panoramas", CVPR Workshops (Image Matching: Local Features & Beyond) 2026.

수작업 검출기

Harris: Harris, C., Stephens, M., "A Combined Corner and Edge Detector", Alvey Vision Conference 1988.

Hessian: Beaudet, P., "Rotationally Invariant Image Operators", ICPR 1978.

SIFT: Lowe, D. G., "Distinctive Image Features from Scale-Invariant Keypoints", IJCV 2004.

KAZE: Alcantarilla, P. F., Bartoli, A., Davison, A. J., "KAZE Features", ECCV 2012.

학습 검출기

FAST: Rosten, E., Drummond, T., "Machine Learning for High-Speed Corner Detection", ECCV 2006.

TILDE: Verdie, Y., Yi, K. M., Fua, P., Lepetit, V., "TILDE: A Temporally Invariant Learned DETector", CVPR 2015.

LIFT: Yi, K. M., Trulls, E., Lepetit, V., Fua, P., "LIFT: Learned Invariant Feature Transform", ECCV 2016.

SuperPoint: DeTone, D., Malisiewicz, T., Rabinovich, A., "SuperPoint: Self-Supervised Interest Point Detection and Description", CVPR Workshops 2018.

LF-Net: Ono, Y., Trulls, E., Fua, P., Yi, K. M., "LF-Net: Learning Local Features from Images", NeurIPS 2018.

평가 데이터·지표

HPatches: Balntas, V., Lenc, K., Vedaldi, A., Mikolajczyk, K., "HPatches: A Benchmark and Evaluation of Handcrafted and Learned Local Descriptors", CVPR 2017.

HardNet: Mishchuk, A., Mishkin, D., Radenović, F., Matas, J., "Working Hard to Know Your Neighbor's Margins: Local Descriptor Learning Loss", NeurIPS 2017.

Mikolajczyk, K., Schmid, C., "A Performance Evaluation of Local Descriptors", TPAMI 2005.

Lenc, K., Vedaldi, A., "Large Scale Evaluation of Local Image Feature Detectors on Homography Datasets", BMVC 2018.